

Guía para Deployar Aplicaciones en AWS — TDS

Guía paso a paso para deployar la aplicación `tds-group-X` en la infraestructura compartida.

IMPORTANTE

- Todos los recursos de la infraestructura se encuentran en la región **US East (Ohio) us-east-2** Asegurarse de realizar todo este proceso y de crear los nuevos recursos teniendo siempre seleccionada esa región en la consola de AWS.
- Todos los recursos creados por ustedes deben tener el prefijo `tds-group-X`. Donde `X` es el número de su grupo.

Tabla de Contenidos

- [Infraestructura Existente](#)
- [Parte 0. Configuración Inicial](#)
- [Parte 1. Crear los Recursos de AWS](#)
- [Parte 2. Crear el Entorno Base](#)
- [Parte 3. Preparar los Manifests de Kubernetes](#)
- [Parte 4. Frontend con S3 + CloudFront](#)
- [Parte 5. CI/CD con GitHub Actions y OIDC](#)
- [Anexo A. Acceso a Recursos de AWS desde Microservicios \(IRSA\)](#)

Infraestructura Existente

Los siguientes recursos ya existen y se comparten, no es necesario crearlos.

Recurso	Detalle
Cuenta AWS + IAM admin	IAM Users para cada grupo con los permisos correspondientes
VPC	2 AZs, 2 subnets públicas, 2 privadas, 1 NAT Gateway
EKS Cluster	<code>tds-cluster</code>
Node Group	<code>tds-nodes</code>
AWS Load Balancer Controller	Corriendo en <code>kube-system</code> , con IRSA
ALB Compartido	IngressGroup <code>tds-shared</code>
Certificado ACM (Ohio)	Wildcard <code>*.tds-linar.udesa.edu.ar</code> para el ALB
Certificado ACM (N. Virginia)	Wildcard <code>*.tds-linar.udesa.edu.ar</code> para CloudFront
Route 53 Hosted Zone	<code>tds-linar.udesa.edu.ar</code>
GitHub OIDC Provider en IAM	<code>token.actions.githubusercontent.com</code> (compartido por todos los equipos)
EKS OIDC Provider en IAM	Para IRSA del ALB Controller y de los ServiceAccounts de cada grupo

Aclaración sobre Bases de Datos Externas (MongoDB Atlas, Supabase, etc.)

Al correr en un cluster EKS sobre private subnets, todo el tráfico de salida de los microservicios hacia internet sale a través de una única IP estática asociada al NAT Gateway de la VPC.

Si la base de datos tiene una "IP Access List" (whitelist de seguridad, como pasa por defecto en Mongo Atlas), hay que agregar la IP pública del NAT Gateway a esa lista. Sino, los microservicios van a recibir un error de conexión (timeout o connection refused) y no van a poder iniciar dentro del entorno K8s.

Para obtener la IP, ir a VPC → NAT Gateways (seleccionar el de la VPC del cluster) → ver la Primary public IPv4 address.

Parte 0. Configuración Inicial

Al recibir sus credenciales IAM, se debe configurar AWS CLI con el profile `tds-group-X`, conectarse al cluster EKS existente, y verificar que la infraestructura funciona antes de arrancar el deploy.

Prerrequisitos. Tener instalados `aws` (AWS CLI v2), `kubectl` y `docker`.

- `aws` es la interfaz de línea de comandos de AWS, la usamos para interactuar con los servicios de AWS desde la terminal.
- `kubectl` es la interfaz de línea de comandos de Kubernetes, la usamos para interactuar con el cluster EKS desde la terminal.

Guías de instalación de [AWS CLI v2](#) y [kubectl](#).

0.1 Login y Configurar MFA

Se les va a dar una URL a la consola de AWS, un IAM Username (ej. `tds-group-1`) y una **contraseña temporal**.

1. Entrar a la URL de la consola y loguearse con el usuario y la contraseña temporal.
2. AWS va a pedir obligatoriamente **cambiar la contraseña**. Ingresar una nueva y guardarla.
3. Una vez en la consola, buscar IAM en la barra superior.
4. En el panel izquierdo, ir a Users y hacer click en el usuario correspondiente (`tds-group-1`).
5. Ir al tab Security credentials.
6. En la sección Multi-factor authentication (MFA), click en Assign MFA device.
7. Elegir un nombre (ej. `celular-grupo1`), seleccionar Authenticator app y seguir los pasos para escanear el QR con Authy o Google Authenticator.

El MFA no es opcional. Es el segundo factor de la consola y de la terminal, porque el paso siguiente usa la sesión de la consola para autenticarse.

0.2 Iniciar Sesión en la Terminal

En vez de generar *access keys* (credenciales de largo plazo que quedan en texto plano en el disco), la CLI se autentica contra la misma sesión de consola del paso anterior y recibe credenciales temporales que se renuevan solas.

```
# 1. Verificar la versión de la CLI, hace falta AWS CLI v2
aws --version

# 2. Fijar la región del profile (esto no guarda credenciales, solo la config)
aws configure set region us-east-2 --profile tds-group-X
aws configure set output json --profile tds-group-X
```

```
# 3. Iniciar sesión. Abre el browser para loguearse con usuario, contraseña y MFA
aws login --profile tds-group-X
```

En WSL (Windows) usar `aws login --remote --profile tds-group-X`. El flujo normal espera que el browser vuelva a un puerto local de la terminal, y el browser de Windows no puede llegar a WSL, así que la página termina en un error 400. Con `--remote` la CLI muestra una URL, se abre en cualquier navegador, y al terminar el login la página da un código para pegar en la terminal. Lo mismo vale para servidores sin entorno gráfico.

Por qué no usamos access keys. Una access key es una credencial de largo plazo que queda guardada en `~/.aws/credentials` sin cifrar y no vence nunca hasta que alguien la borre. Es la causa más común de filtraciones en AWS, se suben a un repo por accidente y quedan publicadas. `aws login` entrega credenciales temporales atadas a la sesión de consola, así que el MFA también protege la terminal y no hay nada permanente que perder. En la propia consola AWS lo desaconseja explícitamente al crear una key.

0.3 Activar el Profile y Verificar la Identidad

```
# Fijar el profile para toda la sesión de terminal
export AWS_PROFILE=tds-group-X

# Verificar que las credenciales funcionan
aws sts get-caller-identity
```

`aws sts get-caller-identity` es el "whoami" de AWS. Llama al servicio STS (Security Token Service) y devuelve la identidad asociada a las credenciales activas. Es el primer comando para confirmar que estás autenticado en la cuenta y usuario correctos.

La salida esperada es similar a esta.

```
{
  "UserId": "AIDXXXXXXXXXXXXXXXXXX",
  "Account": "ACCOUNT_ID",
  "Arn": "arn:aws:iam::ACCOUNT_ID:user/tds-group-0"
}
```

Si da error de credenciales vencidas o inválidas, la sesión caducó y hay que volver a correr `aws login --profile tds-group-X`. Es normal que pase cada tantas horas, y es justamente la ventaja de este método frente a una key permanente.

Si `aws login` funcionó pero después da `InvalidClientTokenId`, en la máquina quedó un profile viejo con el mismo nombre. Las credenciales guardadas en `~/.aws/credentials` tienen prioridad sobre la sesión de `aws login`, así que la CLI sigue usando esas. Abrir `~/.aws/credentials`, borrar el bloque `[tds-group-X]` y reintentar. Con `aws configure list --profile tds-group-X` se ve de dónde está sacando las credenciales.

Si `aws login` no existe, la CLI es vieja. Actualizar a AWS CLI v2 desde la [guía oficial de instalación](#) y reintentar.

0.4 Cheat Sheet de Comandos de Profiles

```
# Ver todos los profiles configurados en la máquina
aws configure list-profiles

# Ver cuál profile está activo en esta sesión
echo $AWS_PROFILE

# Ver la identidad del profile activo
aws sts get-caller-identity

# Activar el profile de TDS para esta sesión de terminal
export AWS_PROFILE=tds-group-X

# Renovar la sesión cuando vence
aws login --profile tds-group-X

# Usar temporalmente otro profile sin cambiar el export
aws <comando> --profile personal-aws

# Ver el detalle de un profile específico
aws configure list --profile tds-group-X
```

`aws configure list-profiles` lista todos los profiles en `~/.aws/config` y `~/.aws/credentials`. Útil cuando tenés profiles de distintos proyectos o cuentas y no te acordás cuál es cuál. El profile `tds-group-X` siempre tiene que aparecer ahí después de hacer el paso 0.2.

0.5 Conectarse al Cluster EKS Existente

```
# Asegurarse de que el profile correcto está activo
export AWS_PROFILE=tds-group-X

# Agregar el cluster al archivo de configuración de kubectl
aws eks update-kubeconfig --region us-east-2 --name tds-cluster
```

¿Qué hace este comando?

Modifica el archivo `~/.kube/config` (la config global de kubectl) y agrega un nuevo `context` apuntando al cluster en AWS. Un `context` tiene tres partes.

Elemento	Qué contiene	De dónde sale
Cluster	URL del API server + certificado CA	Se baja de AWS via <code>eks:DescribeCluster</code>
User	Instrucción para generar tokens temporales (<code>aws eks get-token</code>)	Se genera automáticamente en cada comando <code>kubectl</code>

Elemento	Qué contiene	De dónde sale
Context	Une cluster + user, y se setea como activo	Lo crea el comando

El archivo `~/.kube/config` es global para toda la máquina, un solo archivo que puede tener múltiples clusters. `update-kubeconfig` agrega el cluster de TDS sin tocar los que ya existían (minikube, otros proyectos, etc.).

Si ya tienen otros clusters, se pueden ver todos y switchear entre ellos.

```
# Ver todos los clusters/contextos configurados (* = el activo)
kubectl config get-contexts
# CURRENT    NAME                                     CLUSTER
# *          arn:aws:eks:us-east-2:ACCOUNT_ID:cluster/tds-cluster tds-cluster
#           minikube                               minikube

# Cambiar al cluster de TDS
kubectl config use-context arn:aws:eks:us-east-2:ACCOUNT_ID:cluster/tds-cluster

# Cambiar a minikube para desarrollo local
kubectl config use-context minikube

# Ver cuál es el contexto activo
kubectl config current-context
```

Renombrar el contexto para evitar escribir el ARN completo cada vez.

```
kubectl config rename-context \
  arn:aws:eks:us-east-2:ACCOUNT_ID:cluster/tds-cluster \
  tds-cluster

# Ahora se puede usar el nombre corto
kubectl config use-context tds-cluster
```

0.6 Verificar la Infraestructura Existente

Correr estos 4 comandos y confirmar que todo está en orden antes de arrancar el deploy.

Los comandos 1 y 2 verifican la infraestructura compartida. Si fallan, el problema no es del equipo.

El comando 3 confirma el modelo de permisos. Desde su terminal pueden leer el cluster, pero un `kubectl apply` va a devolver `Forbidden`. Eso es lo esperado, no es un error de configuración. Quien aplica los manifests es el pipeline de GitHub Actions (Parte 5), y los manifests cluster-scoped los aplica el tutor asignado.

```
export AWS_PROFILE=tds-group-X

# 1. Verificar los nodos del cluster
```

```
kubectl get nodes
# Tiene que haber al menos 1 nodo en estado Ready
# NAME                                STATUS  ROLES    AGE  VERSION
# ip-10-XX-XX-XXX.us-east-2.compute.internal  Ready  <none>   Xd   vX.XX.X-eks-XXXXXXX

# 2. Verificar que el ALB Controller está corriendo
kubectl get pods -n kube-system -l app.kubernetes.io/name=aws-load-balancer-controller
# Tiene que haber 1 pod en estado Running

# 3. Confirmar el modelo de permisos, lectura sí y escritura no
kubectl auth can-i get pods --all-namespaces      # -> yes
kubectl auth can-i create deployments -n tds-group-X  # -> no

# 4. Si ya se desplegó la aplicación demo, se la puede mirar
kubectl get pods,ingress -n tds-group-0
# Si tds-group-0 no existe todavía no es un error, la demo aún no está desplegada
```

Parte 1. Crear los Recursos de AWS

1.1 Crear Repositorios ECR

Amazon ECR (Elastic Container Registry) es el private image registry de AWS. Funciona como un "Docker Hub" privado dentro de la cuenta. Cada microservicio necesita su propio repo donde se guardan las versiones (tags) de su imagen Docker.

1. ECR → Repositories → Create repository
2. En Repository name poner `tds-group-X/<service-name>` (ej. `tds-group-1/api-gateway`)
3. En Image tag mutability elegir Mutable, que permite pisar tags como `latest`
4. En Encryption settings dejar los valores por defecto
5. Click en Create repository
6. Repetir para cada microservicio adicional (ej. `tds-group-1/users-service` , `tds-group-1/orders-service`)

La convención de nombres `tds-group-X/<service-name>` permite usar el wildcard `tds-group-X/*` en la IAM policy para restringir el acceso solo a los repositorios de cada equipo.

1.2 Crear Bucket S3 para el Frontend

Amazon S3 (Simple Storage Service) es el object storage de AWS. En esta arquitectura se usa como hosting de los archivos estáticos del frontend (HTML, CSS, JS). El bucket no se expone directamente a internet, CloudFront (nuestra CDN) se encarga de servirlo.

1. S3 → Create bucket. La región no se elige en este formulario, se toma de la que está seleccionada arriba a la derecha en la consola. Verificar que diga US East (Ohio) us-east-2.

Campo	Valor
Bucket type	General purpose
Bucket namespace	Global namespace (la opción por defecto)
Bucket name	<code>tds-group-X-frontend</code>
Block Public Access settings for this bucket	Marcado, Block all public access (el acceso lo maneja CloudFront via OAC)

2. Dejar el resto de las opciones por defecto → Create bucket

No elegir "Account Regional namespace", aunque la consola la marque como recomendada. Esa opción le agrega al nombre un sufijo con el número de cuenta y la región (`tds-group-X-frontend-123456789012-us-east-2-an`), y el resto de la guía asume el nombre exacto `tds-group-X-frontend` . Con el sufijo, la policy del §1.3 no lo reconoce y el pipeline falla al subir el frontend.

1.3 Crear IAM Policy para GitHub Actions

IAM Policies son documentos JSON que definen permisos en AWS. Esta policy autoriza al pipeline de CI/CD (GitHub Actions) a interactuar únicamente con los recursos de este deploy, que son el push de imágenes a ECR, el sync de archivos a S3, la cache invalidation en CloudFront y la conexión al cluster EKS.

1. IAM → Policies → Create policy → tab JSON, y pegar el siguiente documento.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ECRAuth",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "ECRPush",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage",
        "ecr:PutImage",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:DescribeRepositories"
      ],
      "Resource": "arn:aws:ecr:us-east-2:ACCOUNT_ID:repository/tds-group-X/*"
    },
    {
      "Sid": "S3Frontend",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::tds-group-X-frontend",
        "arn:aws:s3:::tds-group-X-frontend/*"
      ]
    },
    {
      "Sid": "CloudFrontInvalidation",
```

```

    "Effect": "Allow",
    "Action": "cloudfront:CreateInvalidation",
    "Resource": "arn:aws:cloudfront::ACCOUNT_ID:distribution/DISTRIBUTION_ID"
  },
  {
    "Sid": "EKSDescribe",
    "Effect": "Allow",
    "Action": "eks:DescribeCluster",
    "Resource": "arn:aws:eks:us-east-2:ACCOUNT_ID:cluster/tds-cluster"
  }
]
}

```

Reemplazar `ACCOUNT_ID` con el Account ID de 12 dígitos, que se puede ver haciendo click en el usuario en la esquina superior derecha de la consola. Los ARNs de ECR y de S3 terminan en `/*`, con barra. Si quedan como `-*`, el pipeline falla con `AccessDenied` al pushear la imagen o al subir el frontend.

Sobre `DISTRIBUTION_ID`. La distribución de CloudFront todavía no existe, se crea en la Parte 4. Por ahora poner `*` en su lugar y volver a este paso al terminar la Parte 4 para reemplazarlo por el ID real (§4.7). Dejarlo en `*` significa que el pipeline de cualquier equipo puede invalidar la distribución de cualquier otro. Funciona, pero no es mínimo privilegio.

2. En Policy name poner `tds-group-X-deploy-policy` → Create policy

1.4 Crear IAM Role con Trust Policy OIDC

OIDC (OpenID Connect) permite que GitHub Actions asuma un IAM Role sin necesidad de guardar access keys como secrets. GitHub emite un token JWT firmado en cada ejecución del workflow y AWS lo valida contra el OIDC provider registrado. Lo que nos permite tener credenciales temporales, sin secrets estáticos y con scope restringido.

1. IAM → Roles → Create role
2. En Trusted entity type elegir Web identity
3. En Identity provider elegir `token.actions.githubusercontent.com`
4. En Audience elegir `sts.amazonaws.com`
5. En GitHub organization poner el nombre exacto de la organización de GitHub del equipo → Next
6. Buscar y marcar `tds-group-X-deploy-policy` (la policy creada en el paso anterior)
7. En la misma pantalla, desplegar Set permissions boundary - optional → elegir Use a permissions boundary to control the maximum role permissions → seleccionar `tds-group-boundary` → Next
8. En Role name poner `GitHubActions-tds-group-X-Deploy` → Create role
9. Copiar el Role ARN (ej. `arn:aws:iam::ACCOUNT_ID:role/GitHubActions-tds-group-X-Deploy`)

¿Qué es la permissions boundary? Es un techo de permisos. Aunque la policy del Role diga "puede todo", el Role nunca puede hacer más de lo que permite la boundary. Así cada equipo puede crear sus propios roles sin poder fabricarse un administrador. Es obligatoria, sin ella la creación del Role falla con `AccessDenied`.

1.5 Crear EKS Access Entry para GitHub Actions

EKS Access Entries mapean identidades IAM a permisos dentro de Kubernetes. Sin este paso, el IAM Role de GitHub Actions puede autenticarse contra el cluster (via `eks:DescribeCluster`) pero no tiene autorización para ejecutar ningún comando `kubectl`. Este access entry le da permisos de edición limitados al namespace `tds-group-X`.

- 1. EKS → Clusters → `tds-cluster` → tab Access → Create access entry
- 2. En IAM principal ARN pegar el ARN de `GitHubActions-tds-group-X-Deploy`
- 3. En Type elegir Standard → Next
- 4. En Add access policy cargar estos valores.

Campo	Valor
Policy name	<code>AmazonEKSEditPolicy</code>
Access scope	Namespaces
Namespaces	Escribir <code>tds-group-X</code> y dar Enter

- 5. Add policy → Create

Parte 2. Crear el Entorno Base

Como los equipos tienen permisos de solo lectura sobre el cluster por motivos de seguridad, deben definir los siguientes manifests YAML base y pushearlos al repositorio. El tutor asignado va a revisar las PRs que agregan manifests de Kubernetes y aplicar manualmente los que lo requieran.

2.1 Crear Namespace con ResourceQuota y LimitRange

Namespaces son divisiones lógicas del cluster de k8s. El ResourceQuota pone un techo máximo global para todo el namespace. El LimitRange actúa por container, impidiendo que se pidan recursos excesivos individualmente.

Crear el archivo `namespace.yaml` en el repo de infraestructura (o en alguno de los repos del equipo) y armar una Pull Request para que el tutor asignado pueda revisar y aplicar el manifest.

```
apiVersion: v1
kind: Namespace
metadata:
  name: tds-group-X
  labels:
    name: tds-group-X
    course: tds-2026
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tds-group-X-quota
  namespace: tds-group-X
spec:
  hard:
    requests.cpu: "2"
    requests.memory: "2Gi"
    limits.cpu: "4"
    limits.memory: "4Gi"
    pods: "8"
    services: "10"
    configmaps: "10"
    secrets: "10"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: tds-group-X
spec:
  limits:
    - min:
```

```
  cpu: "100m"
  memory: "128Mi"
default:
  cpu: "500m"
  memory: "512Mi"
defaultRequest:
  cpu: "100m"
  memory: "128Mi"
max:
  cpu: "500m"
  memory: "512Mi"
type: Container
```

Una vez revisado y aprobado por el tutor asignado, se aplica con permisos de admin (Cluster Scope).

```
kubectl apply -f namespace.yaml
```

Parte 3. Preparar los Manifests de Kubernetes

El equipo debe definir todos los manifests YAML y pushearlos a sus repos. Los manifests deben ser pusheados luego de que el tutor asignado haya aprobado las PRs.

Estructura de carpetas recomendada en el repo

```
├── .github/
│   └── workflows/
│       └── deploy.yml
└── k8s/
    ├── configmap.yaml
    ├── secret.template.yaml ← template con variables, NO el secret real
    ├── deployment.yaml
    ├── service.yaml
    └── ingress.yaml          ← uno solo por equipo, puede vivir en el repo de infra
```

3.1 ConfigMap

Un ConfigMap guarda configuración no sensible (URLs, puertos, feature flags). El pipeline lo aplica automáticamente en cada deploy.

Crear `k8s/configmap.yaml` en cada repo.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-gateway-config
  namespace: tds-group-X
data:
  allowed-origins: "https://tds-group-X.tds-linar.udesa.edu.ar"
  port: "3000"
```

3.2 Secret Template

Los Secrets guardan datos sensibles (passwords, tokens, JWT keys). Nunca se commitean al repo con los valores reales. En cambio, se crea un archivo *template* con variables (`${VARIABLE}`) que el pipeline reemplaza con los valores de GitHub Secrets en runtime.

Crear `k8s/secret.template.yaml` en cada repo.

```
apiVersion: v1
kind: Secret
metadata:
  name: api-gateway-secret
  namespace: tds-group-X
type: Opaque
stringData:
  jwt-secret: "${JWT_SECRET}"
```

3.3 Deployment

Un Deployment gestiona la ejecución de los pods. Define qué imagen usar, cuántas réplicas, los health checks, los resources y cómo hacer rolling updates.

Crear `k8s/deployment.yaml` en cada repo.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
  namespace: tds-group-X
  labels:
    app: api-gateway
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: api-gateway
  template:
    metadata:
      labels:
        app: api-gateway
    spec:
      containers:
        - name: api-gateway
          image: ACCOUNT_ID.dkr.ecr.us-east-2.amazonaws.com/tds-group-X/api-gateway:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3000
          env:
            - name: ALLOWED_ORIGINS
              valueFrom:
                configMapKeyRef:
                  name: api-gateway-config
                  key: allowed-origins
```

```

- name: JWT_SECRET
  valueFrom:
    secretKeyRef:
      name: api-gateway-secret
      key: jwt-secret
  livenessProbe:
    httpGet:
      path: /api/health
      port: 3000
    initialDelaySeconds: 15
    periodSeconds: 10
  readinessProbe:
    httpGet:
      path: /api/health
      port: 3000
    initialDelaySeconds: 5
    periodSeconds: 5
  resources:
    requests:
      cpu: "100m"
      memory: "128Mi"
    limits:
      cpu: "500m"
      memory: "512Mi"

```

Nota. La imagen usa `:latest` para que el pipeline siempre deployee la versión más reciente. El SHA del commit se incluye como tag adicional para trazabilidad y rollback (`image: ...api-gateway:abc1234`).

3.4 Service

Un Service expone los pods bajo una dirección interna estable. El nombre del Service funciona como internal DNS dentro del cluster (ej. `api-gateway:3000`).

Crear `k8s/service.yaml` en el repo.

```

apiVersion: v1
kind: Service
metadata:
  name: api-gateway
  namespace: tds-group-X
spec:
  type: ClusterIP
  selector:
    app: api-gateway
  ports:

```

```
- port: 3000
  targetPort: 3000
```

3.5 Ingress

Un Ingress expone rutas HTTP/HTTPS hacia los services dentro del cluster, operando como un L7 reverse proxy. Como requiere registrar subdominios y routing rules en un cluster compartido, necesita **privilegios de admin**. El equipo define las routing rules y el tutor asignado revisa, aprueba y aplica el manifest.

Crear `k8s/ingress.yaml` en el mismo repo que `k8s/namespace.yaml`.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tds-group-X-ingress
  namespace: tds-group-X
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
    alb.ingress.kubernetes.io/ssl-redirect: "443"
    alb.ingress.kubernetes.io/certificate-arn: ACM_ARN_OHIO
    alb.ingress.kubernetes.io/group.name: tds-shared
    alb.ingress.kubernetes.io/group.order: "200"
spec:
  ingressClassName: alb
  rules:
    - host: tds-group-X.tds-linar.udesa.edu.ar
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port:
                  number: 3000
```

Reemplazar `ACM_ARN_OHIO` con el ARN del certificado wildcard de la región us-east-2. El `group.order` determina la prioridad de evaluación del ALB Controller y debe ser un valor único por equipo (ej. `201` para el grupo 1, `202` para el grupo 2, etc.).

Importante sobre el prefijo `/api/` en el api-gateway. El Ingress está configurado con `path: /api` y `pathType: Prefix`, lo que significa que el ALB solo va a forwardear al api-gateway las requests cuya ruta empiece con `/api/`. Por lo tanto, todos los endpoints del api-gateway deben tener el prefijo `/api/` (ej. `/api/users`, `/api/orders`, `/api/health`). Si la aplicación no sigue esta convención, se recomienda agregar el prefijo `/api` en el router del api-gateway, o en caso de considerarlo mejor, modificar el `path` en el Ingress yaml para que matchee las rutas que el equipo necesita.

Una vez validado y aprobado, el tutor asignado aplica el manifest.

```
kubectl apply -f ingress.yaml
```

3.6 Verificación Post-Deploy (via kubectl)

Una vez que el pipeline de la Parte 5 haya corrido exitosamente, verificar con estos comandos.

```
# Pods corriendo
kubectl get pods -n tds-group-X
# NAME                                READY   STATUS    RESTARTS   AGE
# api-gateway-xxxxx-xxxxx             1/1     Running   0           1m

# Service creado
kubectl get svc -n tds-group-X

# Ingress con ADDRESS (puede tardar 1-2 min)
kubectl get ingress -n tds-group-X
# NAME                                CLASS   HOSTS                                ADDRESS
# tds-group-X-ingress                 alb     tds-group-X.tds-linar.udesa.edu.ar  k8s-tdssshared-
xxxxx...elb.amazonaws.com

# Logs de la aplicación
kubectl logs deployment/api-gateway -n tds-group-X
```

Parte 4. Frontend con S3 + CloudFront

4.1 Crear CloudFront Distribution

Amazon CloudFront es la CDN de AWS, con ~400 edge locations globales. En esta arquitectura sirve los archivos estáticos desde S3 y funciona como reverse proxy para `/api/*` hacia el ALB. Además maneja los certificados SSL y la terminación HTTPS.

1. CloudFront → Distributions → Create distribution
2. En Step 1 - Choose a plan, seleccionar el plan Free (\$0).
3. En Step 2 - Get started completar
 - En Distribution name poner `tds-group-X-frontend`
 - En Distribution type elegir Single website or app
 - Domain (Route 53 managed) se deja en blanco.
4. En Step 3 - Specify origin completar
 - En Origin type seleccionar Amazon S3.
 - En S3 origin buscar y seleccionar el bucket `tds-group-X-frontend.s3.us-east-2.amazonaws.com`.
 - Origin path se deja vacío.
 - En Settings marcar Allow private S3 bucket access to CloudFront - Recommended. Esto crea el OAC y actualiza la bucket policy de S3 automáticamente. Dejar el resto en los valores default.
5. En Step 4 - Enable security completar
 - En Web Application Firewall (WAF) dejar todo default, sin marcar `Use monitor mode`. AWS provee protecciones L7 sin costo adicional.
6. Click en Next y luego en Create distribution.

4.2 Configurar CNAME, Certificado SSL y Default Root Object

Con la distribución ya creada, seguir estos pasos.

1. Dentro del panel de la distribución, asegurarse de estar en el tab General.
2. En la sección Settings, click en Edit.
3. Completar los siguientes campos.
 - En Alternate domain name (CNAME) poner `tds-group-X.tds-linar.udesa.edu.ar`
 - En Custom SSL certificate desplegar la lista y seleccionar el certificado wildcard de Virginia, el que empieza con `*`. (`*.tds-linar.udesa.edu.ar`).
 - En Default root object poner `index.html`
 - Dejar el resto en default.
4. Bajar hasta el final y click en Save changes.

Cuidado al elegir el certificado. En la lista aparecen dos entradas parecidas para el mismo dominio, `*.tds-linar.udesa.edu.ar` y `tds-linar.udesa.edu.ar` (sin el asterisco). Las dos funcionan hoy, pero la segunda es de una edición anterior de la materia y vence en octubre de 2026.

4.3 Agregar Origin del ALB

1. CloudFront → la distribución de tds-group-X → tab Origins → Create origin

Campo	Valor
Origin domain	El DNS del ALB (<code>k8s-tdssshared-xxxxx.us-east-2.elb.amazonaws.com</code>)
Protocol	HTTPS only
Name	<code>alb-backend</code>

El DNS del ALB está en EC2 → Load balancing → Load balancers → seleccionar el único ALB → DNS name.

2. Create origin

4.4 Crear Behavior para `/api/*` (o la ruta requerida por el equipo)

1. Tab Behaviors → Create behavior

Campo	Valor
Path pattern	<code>/api/*</code>
Origin	<code>alb-backend</code>
Viewer protocol policy	HTTPS only
Allowed HTTP methods	GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE
Cache key and origin requests	Cache policy and origin request policy (recommended)
Cache policy	CachingDisabled (managed). <code>/api/*</code> no se cachea.
Origin request policy	AllViewer (managed, forwarda el Host header)

2. Create behavior

4.5 Servir las Rutas del SPA (CloudFront Function)

Un SPA resuelve el ruteo en el browser, así que `https://tds-group-X.tds-linar.udesa.edu.ar/Login` no existe como objeto en S3. Si no se hace nada, S3 devuelve 403 AccessDenied (con OAC y bucket privado, una key inexistente da 403, no 404) y el usuario ve un error de CloudFront en lugar de la aplicación.

No usar las "Custom error responses" de la distribución para esto. Esa configuración es a nivel distribución y se aplica a todos los behaviors, incluido `/api/*`. Un 403 o un 404 legítimo de la API quedaría reescrito a `/index.html` con status 200, y el frontend recibiría HTML donde espera JSON. El manejo de errores de la aplicación deja de funcionar en silencio. Por ejemplo, un login que devuelve 403 porque el mail no está verificado le llegaría al browser como un 200 exitoso.

La solución correcta es una CloudFront Function asociada únicamente al behavior default. Las Functions se asocian por behavior, así que `/api/*` no se ve afectado.

1. CloudFront → panel izquierdo → Functions → Create function
2. En Name poner `tds-group-X-spa-rewrite` y en Runtime elegir `cloudfront-js-2.0` → Create function
3. En Function code, reemplazar todo el contenido por el siguiente código.

```
function handler(event) {
  var request = event.request;
  var uri = request.uri;

  // Si la ruta no tiene extensión, es una ruta del SPA (/login, /dashboard, ...).
  // Se sirve index.html y el router de React resuelve el resto en el browser.
  if (uri.indexOf('.') === -1) {
    request.uri = '/index.html';
  }

  return request;
}
```

4. Save changes → tab Publish → Publish function
5. Tab Associate → Add association, con estos valores.

Campo	Valor
Distribution	La distribución de <code>tds-group-X</code>
Event type	Viewer request
Cache behavior	Default (*)

6. Add association

Importante. La asociación va en el behavior Default (*), nunca en `/api/*`. Si se asocia a `/api/*`, las llamadas a la API se reescriben a `/index.html` y el backend deja de recibirlas.

Verificación, una vez que el frontend esté deployado (Parte 5).

```
# Una ruta del SPA devuelve la aplicación
curl -s -o /dev/null -w "%{http_code}\n" \
  https://tds-group-X.tds-linar.udesa.edu.ar/login
# -> 200
```

```
# Una ruta inexistente de la API sigue devolviendo el error real del backend
curl -i https://tds-group-X.tds-linar.udesa.edu.ar/api/no-existe
# -> 404 del backend, NO un 200 con el HTML del index
```

4.6 Crear Registro DNS

Amazon Route 53 es el servicio de DNS de AWS. Un record tipo A con Alias permite apuntar un subdominio (`tds-group-X.tds-linar.udesa.edu.ar`) directamente a un recurso de AWS (en este caso, la distribución CloudFront) sin necesidad de conocer la IP.

1. Route 53 → Hosted zones → `tds-linar.udesa.edu.ar` → Create record

Campo	Valor
Record name	<code>tds-group-X</code>
Record type	A
Alias	Activar
Route traffic to	Alias to CloudFront distribution
Distribution	Seleccionar la distribución de <code>tds-group-X</code>

2. Create records

4.7 Ajustar la Policy de CloudFront (mínimo privilegio)

Ahora que la distribución existe, volver a la policy creada en §1.3 y reemplazar el comodín por el ID real.

1. Copiar el Distribution ID (ej. `E1A2B3C4D5E6F7`) del panel de la distribución en CloudFront.
2. IAM → Policies → `tds-group-X-deploy-policy` → Edit → tab JSON.
3. En el statement `CloudFrontInvalidation` , cambiar el `Resource` .

```
"Resource": "arn:aws:cloudfront::ACCOUNT_ID:distribution/E1A2B3C4D5E6F7"
```

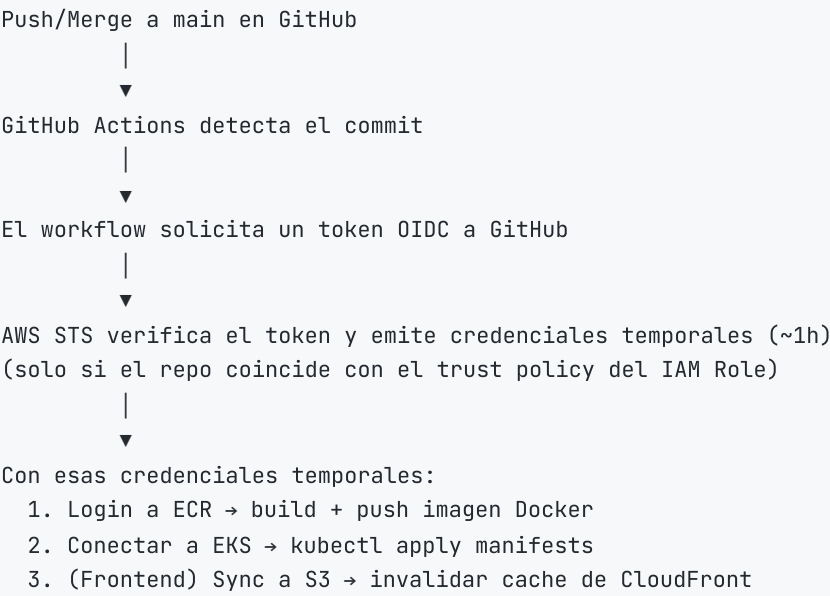
4. Next → Save changes.

Con el comodín `distribution/*` , el role de un equipo puede invalidar la caché de la distribución de cualquier otro equipo de la cuenta. Con el ID explícito, solo la propia. Es el mismo criterio que se aplicó a ECR (`tds-group-X/*`) y a S3.

Parte 5. CI/CD con GitHub Actions y OIDC

Mediante OIDC, los workflows asumen el IAM Role creado en el paso 1.4 sin necesidad de guardar access keys como secrets. El pipeline se encarga automáticamente de buildear la imagen Docker, pushearla a ECR, y aplicar los manifests de K8s.

¿Cómo funciona el flujo?



5.1 Configurar GitHub Secrets

En cada repositorio del equipo → Settings → Secrets and variables → Actions → New repository secret.

Secrets compartidos por todos los repositorios del equipo

Secret	Valor	Descripción
AWS_REGION	us-east-2	Región de AWS
AWS_ACCOUNT_ID	ACCOUNT_ID	Account ID de 12 dígitos
EKS_CLUSTER_NAME	tds-cluster	Nombre del cluster
AWS_ROLE_ARN	arn:aws:iam::ACCOUNT_ID:role/GitHubActions-tds-group-X-Deploy	IAM Role ARN creado en el paso 1.4

Secrets específicos de cada servicio (varían según la aplicación)

Secret	Ejemplo	Descripción
<code>DATABASE_URL</code>	<code>postgresql://user:pass@host:5432/db</code>	Connection string de la base de datos
<code>JWT_SECRET</code>	<code>clave-random-larga-minimo-32-chars</code>	Secret para firmar JWTs
<code>CLOUDFRONT_DISTRIBUTION_ID</code>	<code>EXXXXXXXX</code>	Solo para el repositorio de frontend

5.2 Workflow de Backend

Crear `.github/workflows/deploy.yml` en cada repo de microservicio.

```

name: Deploy to EKS

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read

    env:
      AWS_REGION: ${ secrets.AWS_REGION }
      AWS_ACCOUNT_ID: ${ secrets.AWS_ACCOUNT_ID }
      EKS_CLUSTER_NAME: ${ secrets.EKS_CLUSTER_NAME }
      IMAGE: ${ secrets.AWS_ACCOUNT_ID }.dkr.ecr.${ secrets.AWS_REGION }.amazonaws.com/tds-
group-X/api-gateway

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          role-session-name: gha-tds-group-X-api-gateway
          aws-region: ${ secrets.AWS_REGION }

      - name: Login to Amazon ECR
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build and push Docker image
        run: |

```

```

    docker build -t $IMAGE:latest -t $IMAGE:${{ github.sha }} .
    docker push $IMAGE:latest
    docker push $IMAGE:${{ github.sha }}

- name: Install kubectl
  uses: azure/setup-kubectl@v4

- name: Configure kubectl for EKS
  run: |
    aws eks update-kubeconfig --region $AWS_REGION --name $EKS_CLUSTER_NAME

- name: Render Secret from template
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }
    JWT_SECRET: ${ secrets.JWT_SECRET }
  run: |
    envsubst < k8s/secret.template.yaml > k8s/secret.yaml

- name: Apply Kubernetes manifests
  run: |
    kubectl apply -f k8s/configmap.yaml
    kubectl apply -f k8s/secret.yaml
    kubectl apply -f k8s/deployment.yaml
    kubectl apply -f k8s/service.yaml

- name: Restart deployment
  run: |
    kubectl rollout restart deployment/api-gateway -n tds-group-X
    kubectl rollout status deployment/api-gateway -n tds-group-X --timeout=180s

```

¿Qué hace cada paso?

Step	Qué hace
<code>permissions.id-token:</code> <code>write</code>	Habilita OIDC y permite solicitar el token JWT a GitHub
<code>configure-aws-credentials@v4</code>	Asume el IAM Role vía OIDC. AWS devuelve credenciales temporales
<code>amazon-ecr-login@v2</code>	Ejecuta <code>docker login</code> a ECR con las credenciales temporales
<code>docker build + push</code>	Bildea la imagen con dos tags, <code>latest</code> y el SHA del commit (para rollback)
<code>update-kubeconfig</code>	Configura <code>kubectl</code> para conectar al cluster EKS
<code>envsubst</code>	Reemplaza <code>\${VARIABLE}</code> en el template con los valores de GitHub Secrets. El <code>secret.yaml</code> resultante vive solo en el runner

Step	Qué hace
<code>kubectl apply</code>	Aplica configmap, secret, deployment y service. No aplica <code>namespace.yaml</code> ni <code>ingress.yaml</code> , que van por PR y los aplica el tutor asignado (§2 y §3.5)
<code>rollout restart</code>	Fuerza a K8s a recrear los pods con la imagen nueva y espera a que el deploy termine

Reemplazar `tds-group-X` (namespace y ECR prefix), `api-gateway` (nombre del servicio), y las env vars en "Render Secret" con los secrets que el servicio necesite.

5.3 Workflow de Frontend

```
name: Deploy Frontend

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          role-session-name: gha-tds-group-X-frontend
          aws-region: ${ secrets.AWS_REGION }

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build
```

```

run: npm run build
env:
  VITE_API_URL: https://tds-group-X.tds-linar.udesa.edu.ar

- name: Sync to S3
  run: |
    aws s3 sync dist/ s3://tds-group-X-frontend/ --delete

- name: Invalidate CloudFront cache
  run: |
    aws cloudfront create-invalidation \
      --distribution-id ${ secrets.CLOUDFRONT_DISTRIBUTION_ID } \
      --paths "/*"

```

Reemplazar `tds-group-X`, la carpeta de build (`dist/` para Vite/Vue, `build/` para Create React App), y las env vars del build.

5.4 Orden del Primer Deploy

La primera vez, ejecutar el trigger manual en este orden (en cada repo → tab **Actions** → **Run workflow**).

1. Servicios internos, porque el API Gateway depende de ellos
2. API Gateway, que referencia los servicios internos por DNS de K8s
3. Frontend, que referencia el subdominio que ya debe estar resolviendo

5.5 Cómo Actualizar la Aplicación

Si se modificó código (frontend o microservicio)

- Merge de una PR a `main` → GitHub Actions se ejecuta automáticamente

Si se modificó un ConfigMap (nueva URL, nuevo CORS origin)

- Editar `k8s/configmap.yaml` → push a `main` → el workflow aplica el cambio y reinicia el deployment

Si se modificó un Secret (nueva contraseña de base de datos)

1. Actualizar el valor en GitHub Secrets (Settings → Secrets → editar)
2. Re-ejecutar el workflow manualmente desde la tab **Actions** → **Run workflow**

Si se necesita agregar un nuevo microservicio

1. Crear un nuevo repo en ECR llamado `tds-group-X/nuevo-servicio`
2. Crear el repo en GitHub con `Dockerfile`, `k8s/deployment.yaml`, `k8s/service.yaml`, `k8s/configmap.yaml`, `k8s/secret.template.yaml`, `.github/workflows/deploy.yaml`

Anexo A. Acceso a Recursos de AWS desde Microservicios (IRSA)

Caso de uso. Uno o más microservicios necesitan leer y/o escribir objetos en un bucket S3 en runtime (ej. subir imágenes de perfil, guardar archivos generados, servir assets privados).

La solución es IRSA (IAM Roles for Service Accounts). IRSA vincula un IAM Role a un Kubernetes ServiceAccount, con tres garantías.

- Solo los pods que usen ese ServiceAccount reciben credenciales.
- Las credenciales son **temporales** (tokens STS inyectados automáticamente via webhook).
- Se respeta el principio de least privilege, porque cada microservicio accede **únicamente** a su bucket.

Referencia. La demo en `tds-group-0` ya usa este patrón para acceder a SQS (`tds-group-0-sa` → `tds-group-0-sqs-role`).

A.1 Crear el Bucket S3 para Assets

Si el equipo necesita un bucket dedicado para archivos de la aplicación (distinto al bucket del frontend).

1. S3 → Create bucket, con la región US East (Ohio) us-east-2 seleccionada en la consola.

Campo	Valor
Bucket type	General purpose
Bucket namespace	Global namespace
Bucket name	<code>tds-group-X-assets</code>
Block Public Access	Marcado, Block all public access

2. Dejar el resto por defecto → Create bucket

Si los archivos necesitan ser públicos (ej. imágenes de perfil), se pueden servir con una distribución CloudFront + OAC apuntando a este bucket, igual que con el frontend.

A.2 Crear IAM Policy para Acceso a S3

1. IAM → Policies → Create policy → tab JSON, y pegar el siguiente documento.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {

```

```

    "Sid": "S3ReadWriteAssets",
    "Effect": "Allow",
    "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:ListBucket"
    ],
    "Resource": [
        "arn:aws:s3:::tds-group-X-assets",
        "arn:aws:s3:::tds-group-X-assets/*"
    ]
}
]
}

```

Ajustar las actions según la necesidad. Si el microservicio solo necesita leer, sacar `s3:PutObject` y `s3:DeleteObject` .

2. En Policy name poner `tds-group-X-s3-assets-policy` → Create policy

A.3 Crear IAM Role con Trust Policy EKS OIDC

A diferencia del Role de GitHub Actions (que confía en el OIDC provider de GitHub), este Role confía en el OIDC provider del cluster EKS. Esto permite que los pods asuman el Role directamente.

1. IAM → Roles → Create role
2. En Trusted entity type elegir Custom trust policy
3. Pegar la siguiente Trust Policy.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::ACCOUNT_ID:oidc-provider/oidc.eks.us-east-2.amazonaws.com/id/EKS_OIDC_ID"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "oidc.eks.us-east-2.amazonaws.com/id/EKS_OIDC_ID:sub": "system:serviceaccount:tds-group-X:tds-group-X-s3-sa",
          "oidc.eks.us-east-2.amazonaws.com/id/EKS_OIDC_ID:aud": "sts.amazonaws.com"
        }
      }
    }
  ]
}

```

```
]
}
```

Qué reemplazar

- `ACCOUNT_ID` con el Account ID de 12 dígitos.
- `EKS_OIDC_ID` con el ID del OIDC provider del cluster. Se saca de EKS → Clusters → `tds-cluster` → tab Overview → copiar el segmento final (después del último `/`) de la OpenID Connect provider URL.
- `tds-group-X` con el namespace y nombre del ServiceAccount del equipo.

La condición `StringEquals` en `sub` restringe el Role a un único ServiceAccount en un único namespace. Ningún otro pod del cluster puede asumir este Role.

4. Click en Next.
5. Buscar y marcar `tds-group-X-s3-assets-policy` (la policy creada en A.2).
6. Desplegar **Set permissions boundary - optional** y seleccionar `tds-group-boundary`, igual que en el §1.4 → Next.
7. En Role name poner `tds-group-X-s3-role` → Create role.
8. Copiar el Role ARN (ej. `arn:aws:iam::ACCOUNT_ID:role/tds-group-X-s3-role`).

A.4 Crear el ServiceAccount en Kubernetes

Crear `k8s/service-account.yaml` en el repo del microservicio que necesita acceso a S3.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: tds-group-X-s3-sa
  namespace: tds-group-X
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT_ID:role/tds-group-X-s3-role
```

Una vez revisado y aprobado el manifest por el tutor asignado, se aplica con permisos de admin (Cluster Scope).

```
kubectl apply -f service-account.yaml
```

El webhook de IRSA en EKS intercepta la creación de pods que referencian este ServiceAccount e inyecta automáticamente tres cosas.

- Una env var `AWS_ROLE_ARN` con el ARN del Role.
- Una env var `AWS_WEB_IDENTITY_TOKEN_FILE` apuntando al JWT token montado en el pod.
- Un projected volume con el token OIDC del ServiceAccount.

Los AWS SDKs (Node.js, Python, Java, Go, etc.) detectan estas variables automáticamente y ejecutan `AssumeRoleWithWebIdentity` de forma transparente. No se necesita código adicional para autenticarse, alcanza con llamar a los métodos de S3 del SDK.

A.5 Vincular el ServiceAccount al Deployment

Modificar `k8s/deployment.yaml` para que los pods usen el ServiceAccount con permisos de S3.

```
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: mi-servicio
  template:
    metadata:
      labels:
        app: mi-servicio
    spec:
      serviceAccountName: tds-group-X-s3-sa    # ← el ServiceAccount de A.4
      containers:
        - name: mi-servicio
          image: ACCOUNT_ID.dkr.ecr.us-east-2.amazonaws.com/tds-group-X/mi-servicio:latest
          # resto de la configuración
```

Importante. Solo agregar `serviceAccountName` al Deployment del microservicio que efectivamente necesita acceso a S3. Los demás Deployments que no lo necesiten deben seguir usando el ServiceAccount default del namespace.

A.6 Agregar el Bucket Name al ConfigMap

Para que el microservicio conozca el bucket name sin hardcodearlo, agregarlo al ConfigMap.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mi-servicio-config
  namespace: tds-group-X
data:
  s3-bucket-name: "tds-group-X-assets"
  s3-region: "us-east-2"
  # ... otras configuraciones del servicio ...
```

Y referenciar las env vars en el Deployment.

```
env:
  - name: S3_BUCKET_NAME
    valueFrom:
      configMapKeyRef:
        name: mi-servicio-config
```

```
        key: s3-bucket-name
-   name: S3_REGION
    valueFrom:
      configMapKeyRef:
        name: mi-servicio-config
        key: s3-region
```

A.7 Verificación

Una vez aplicados todos los manifests, verificar que IRSA está correctamente inyectado.

```
# 1. Verificar que el ServiceAccount existe y tiene la annotation
kubectl get sa tds-group-X-s3-sa -n tds-group-X -o yaml
# Tiene que mostrar esta annotation
#   annotations:
#     eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT_ID:role/tds-group-X-s3-role

# 2. Verificar que el pod tiene las variables de entorno inyectadas
kubectl exec deployment/mi-servicio -n tds-group-X -- env | grep AWS
# Salida esperada
# AWS_ROLE_ARN=arn:aws:iam::ACCOUNT_ID:role/tds-group-X-s3-role
# AWS_WEB_IDENTITY_TOKEN_FILE=/var/run/secrets/eks.amazonaws.com/serviceaccount/token

# 3. Verificar acceso a S3 desde dentro del pod
kubectl exec deployment/mi-servicio -n tds-group-X -- \
  aws s3 ls s3://tds-group-X-assets/ 2>&1
# Si el contenedor tiene AWS CLI instalado, debería listar el contenido del bucket.
# De lo contrario, verificar mediante los logs de la aplicación.
```